

Question 1

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier, AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load dataset
data = pd.read_csv('Dataset_Day11.csv')
print("Dataset Loaded Successfully.\n")
X = data.drop('Outcome', axis=1)
y = data['Outcome']

missing_cols = ['Glucose', 'BloodPressure', 'BMI', 'DiabetesPedigreeFunction']

# Replace 0s with median (excluding zeros)
for col in missing_cols:
    median_val = data[data[col] != 0][col].median()
    data[col] = data[col].replace(0, median_val)
print(data[missing_cols].describe())
print("Missing data was found in four columns. These were replaced with median values, " \
      "which is robust to outliers and maintains the central tendency.")
```

```
Dataset Loaded Successfully,
count      Glucose  BloodPressure      BMI  DiabetesPedigreeFunction
mean    121.656250    72.386719    32.455208      0.471876
std      30.438286    12.096642     6.875177      0.331329
min       44.000000    24.000000    18.200000      0.078000
25%       99.750000    64.000000    27.500000      0.243750
50%      117.000000    72.000000    32.300000      0.372500
75%      140.250000    80.000000    36.600000      0.626250
max      199.000000   122.000000    67.100000      2.420000
Missing data was found in four columns. These were replaced with median values, which is robust to outliers and maintains the c
cy.
```

Question 2

```
def remove_outliers_iqr(dataframe, columns):
    df_clean = dataframe.copy()
    for col in columns:
        Q1 = df_clean[col].quantile(0.25)
        Q3 = df_clean[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        df_clean = df_clean[(df_clean[col] >= lower) & (df_clean[col] <= upper)]
    return df_clean

numeric_cols = data.columns.drop('Outcome')
df_clean = remove_outliers_iqr(data, numeric_cols)

# Show number of rows before and after outlier removal
print(f"Original dataset shape: {data.shape}")
print(f"Dataset shape after outlier removal: {df_clean.shape}")
print(f"Number of rows removed as outliers: {data.shape[0] - df_clean.shape[0]}")
print("Outliers can skew distance calculations in kNN. Removing them ensures better model stability and less variance
```

```
Original dataset shape: (768, 7)
Dataset shape after outlier removal: (698, 7)
Number of rows removed as outliers: 70
Outliers can skew distance calculations in kNN. Removing them ensures better model stability and less variance in prediction.
```

Question 3 a

```
# Split into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("Data split into 80% training and 20% testing sets.\n")

# Train Decision Tree Classifier with default parameters
clf_default = DecisionTreeClassifier(random_state=42)
clf_default.fit(X_train, y_train)
y_pred_default = clf_default.predict(X_test)
print("Default Decision Tree model trained.\n")

#Bagging with Decision Trees
bagging_f1 = []
bagging_acc = []

for n in range(2, 26):
    bag_model = BaggingClassifier(
        estimator=DecisionTreeClassifier(),
        random_state=42
    )

    bag_model.fit(X_train, y_train)
    y_pred = bag_model.predict(X_test)

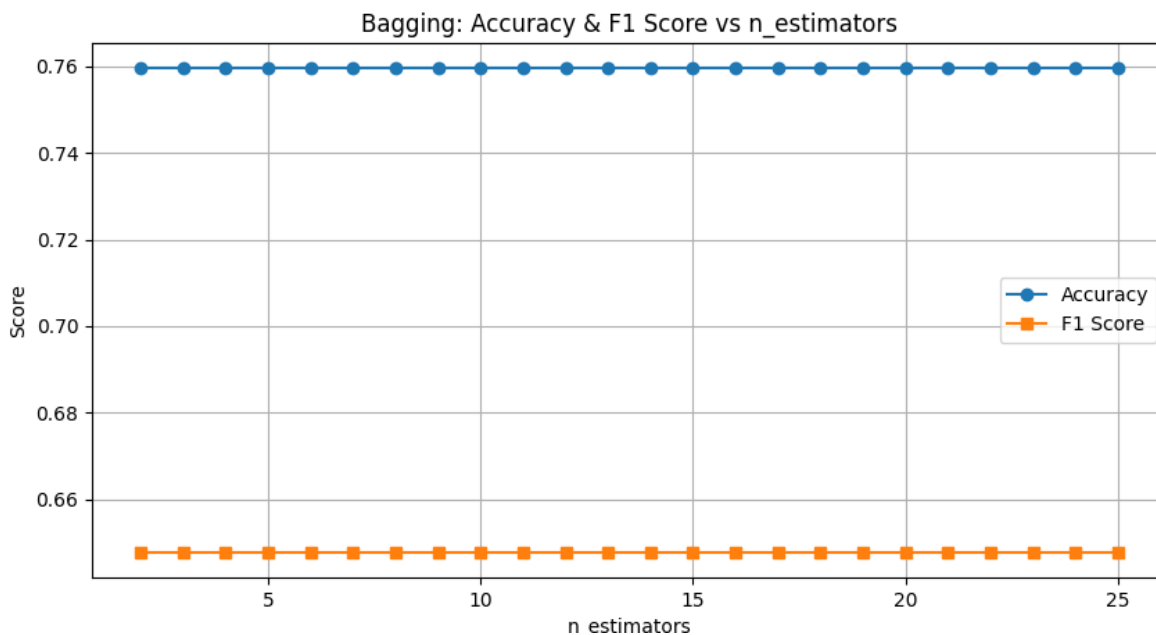
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    bagging_f1.append(f1)
    bagging_acc.append(acc)

    if n == 10: # Example: print for a particular n
        print("Bagging Classifier (n_estimators=10):")
        print(f"Accuracy: {acc:.4f}, Precision: {prec:.4f}, Recall: {rec:.4f}, F1 Score: {f1:.4f}")

# Plot
plt.figure(figsize=(10, 5))
plt.plot(range(2, 26), bagging_acc, label='Accuracy', marker='o')
plt.plot(range(2, 26), bagging_f1, label='F1 Score', marker='s')
plt.title("Bagging: Accuracy & F1 Score vs n_estimators")
plt.xlabel("n_estimators")
plt.ylabel("Score")
plt.legend()
plt.grid(True)
plt.show()
```

```
Bagging Classifier (n_estimators=10):
Accuracy: 0.7597, Precision: 0.6800, Recall: 0.6182, F1 Score: 0.6476
```



Question 3b

```
#Random Forest
rf_metric = []
for n in range(2, 26):
    rf_model = RandomForestClassifier(n_estimators=n, random_state=42)
    rf_model.fit(X_train, y_train)
    y_pred = rf_model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

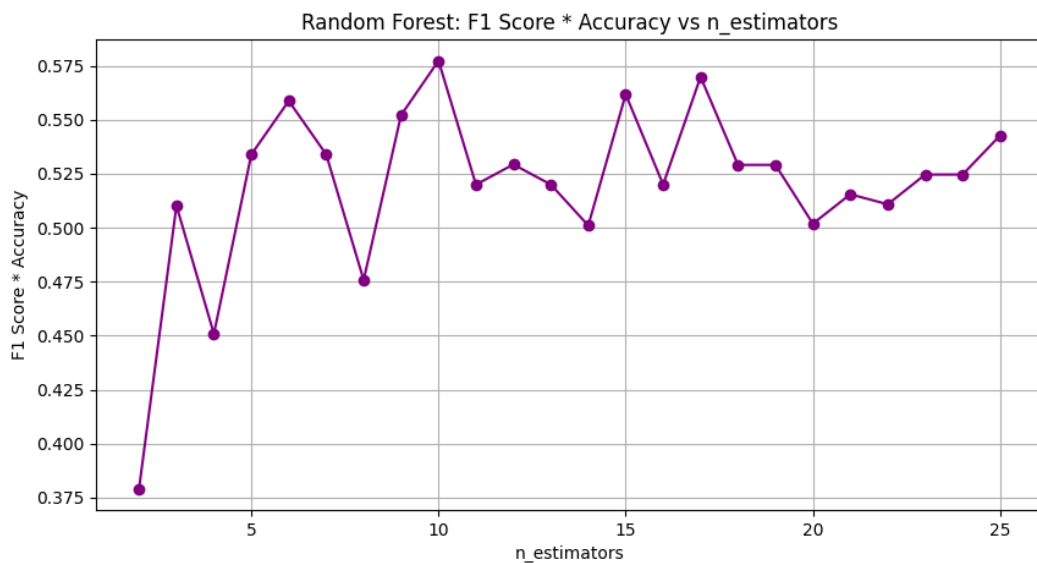
    rf_metric.append(f1 * acc)

if n == 10: # Example: print for a particular
    print("Random Forest (n_estimators=10):")
    print(f"Accuracy: {acc:.4f}, Precision: {prec:.4f}, Recall: {rec:.4f}, F1 Score: {f1:.4f}")

# Plot
plt.figure(figsize=(10, 5))
plt.plot(range(2, 26), rf_metric, marker='o', color='purple')
plt.title("Random Forest: F1 Score * Accuracy vs n_estimators")
plt.xlabel("n_estimators")
plt.ylabel("F1 Score * Accuracy")
plt.grid(True)
plt.show()
```

Random Forest (n_estimators=10):

Accuracy: 0.8052, Precision: 0.7451, Recall: 0.6909, F1 Score: 0.7170



Question 3c

```
#AdaBoost
adaboost_model = AdaBoostClassifier(
    base_estimator=DecisionTreeClassifier(),
    n_estimators=50, # default
    random_state=42
)
adaboost_model.fit(X_train, y_train)
y_pred = adaboost_model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("AdaBoost Classifier:")
print(f"Accuracy: {acc:.4f}, Precision: {prec:.4f}, Recall: {rec:.4f}, F1 Score: {f1:.4f}")
```

AdaBoost Classifier:

Accuracy: 0.7338, Precision: 0.6250, Recall: 0.6364, F1 Score: 0.6306